ChatSphere

Backend Engineering Architecture & Learning Guide

Learn Backend Development Through a Real-Time Chat Application

Purpose

This PDF is a beginner-friendly backend development book built around the current ChatSphere source code. It rewrites and reorganizes the existing Markdown guide instead of simply printing it.

Use this book for	How it helps
Learning backend development	Introduces terms before using them heavily.
Understanding ChatSphere	Maps concepts to real source files and flows.
Revision and interviews	Includes diagrams, cheat sheet, glossary, roadmap, and questions.
Future engineering work	Separates current behavior from production-scale alternatives.

Table of Contents

Chapter 1 - What Actually Is A Backend?

What is it?

A backend is the server-side part of an application. It receives requests, checks identity and permissions, validates data, talks to databases and external services, and returns responses.

Why do we need it?

ChatSphere needs a backend because messages, users, groups, calls, files, statuses, stars, reactions, and security rules must survive page refreshes and cannot be trusted to the browser alone.

How does it work?

The frontend sends an HTTP request or WebSocket event. The backend routes it, authenticates it, validates input, calls the store layer, writes or reads PostgreSQL, then returns JSON or broadcasts realtime events.

How ChatSphere uses it

When User A sends Hello, AppShell sends POST /api/v1/messages or a message_send WebSocket event. Gin routes it, requireUser identifies the sender, SaveMessage inserts it, and the realtime hub broadcasts chat.message.

What would happen without it?

The app would be local-only. A page refresh could lose data, users could edit browser code to bypass rules, and recipients would not reliably receive messages.

What should I learn?

HTTP, APIs, endpoints, JSON, request/response, status codes, handlers, persistence, and realtime broadcasts.

Where is it in the code?

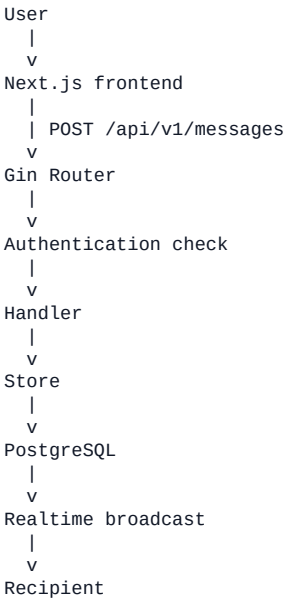
• frontend/src/components/AppShell.tsx

• backend/internal/http/router.go

• backend/internal/store/store.go

• backend/internal/realtime/client.go

• backend/internal/realtime/hub.go



Term	Meaning	ChatSphere example
HTTP	Protocol for request/response	POST /api/v1/messages

Term	Meaning	ChatSphere example
API	A callable contract	/api/v1 routes
Endpoint	Method plus path	PATCH /api/v1/messages/:id
JSON	Text data format	{"body":"Hello"}
Status code	Numeric outcome	200, 401, 403, 500

Chapter 2 - ChatSphere Architecture

What is it?

Architecture is the high-level shape of the system: components, responsibilities, data flow, and external dependencies.

Why do we need it?

A realtime chat app has more moving parts than a simple form app. It needs frontend UI, backend APIs, durable storage, realtime delivery, media handling, calling, email, AI, and deployment configuration.

How does it work?

The browser or Android WebView runs the Next.js frontend. The frontend calls the Go backend over HTTP and WebSocket. The backend uses Gin handlers, store methods, PostgreSQL, Cloudinary, Brevo, and Gemini.

How ChatSphere uses it

The backend owns API security and data. The frontend owns presentation, input, optimistic UI, WebRTC browser APIs, and user interaction.

What would happen without it?

All code would become tangled. The frontend might handle secret logic, or the backend might become a giant file with routing, SQL, realtime, and providers mixed together.

What should I learn?

Client/server boundaries, responsibility separation, external service boundaries, and why diagrams help you reason about systems.

Where is it in the code?

- backend/cmd/server/main.go
- backend/internal/config/config.go
- backend/internal/http/router.go
- backend/internal/store/store.go
- frontend/src/lib/webrtcConfig.ts
- ChatSphere_Android/app/src/main/java/com/chatsphere/app/MainActivity.kt

```
Browser / Android WebView
        |
        v
Next.js frontend
        |
        +-- HTTP REST API ----> Go/Gin backend ---> Store ---> PostgreSQL
        |
        +-- WebSocket -------> Realtime Hub
        |
        +-- WebRTC media ----> Peer-to-peer or TURN relay

Backend external services:
  Cloudinary - media storage/CDN
  Brevo      - OTP email
  Gemini     - AI assistant

Frontend/WebRTC services:
  Google STUN
  Metered TURN when configured
```

Frontend responsibility	Backend responsibility
Render UI and optimistic state	Authenticate and authorize
Open WebSocket	Track connections and broadcast events
Create RTCPeerConnection	Relay signaling and call session state
Compress/select files	Validate, store, and serve media

Frontend responsibility	Backend responsibility
Show errors	Return safe status codes

Chapter 3 - Go For Backend Development

What is it?

Go is the language used by the ChatSphere backend. It is compiled, typed, and has built-in concurrency tools.

Why do we need it?

Backends need predictable performance, clear error handling, and the ability to handle many users at once. Go is a good fit for HTTP servers and realtime connection handling.

How does it work?

Go code is organized into packages. Data is represented with structs. Methods attach behavior to structs. Errors are returned explicitly. Goroutines run concurrent work. Channels and mutexes coordinate shared state.

How ChatSphere uses it

ChatSphere uses packages such as http, store, realtime, config, and gemini. Store structs model users/messages/groups. The realtime hub uses goroutines, channels, maps, and RWMutex.

What would happen without it?

A backend can be built in other languages, but in this project you would not understand routing, persistence, auth, or realtime behavior without Go fundamentals.

What should I learn?

packages, functions, structs, methods, interfaces, pointers, maps, slices, errors, defer, context, time.Time, JSON tags, goroutines, channels, mutexes.

Where is it in the code?

• backend/internal/store/store.go

• backend/internal/realtime/hub.go

• backend/internal/realtime/client.go

• backend/internal/http/ai.go

Concept	Tiny meaning	ChatSphere example
package	Code namespace	store, realtime, config
struct	Fields grouped together	User, Message, Hub
method	Function on a type	Store.SaveMessage
interface	Required behavior	geminiCaller
pointer	Reference to value	*Store, *Hub
map	Key/value lookup	online map
slice	List	[]Message
error	Failure value	return User{}, err
goroutine	Concurrent work	go hub.Run()
channel	Goroutine pipe	broadcast chan Event
mutex	Lock	Hub.mu

```
type Hub struct {
    clients    map[*Client]bool
    register   chan *Client
    unregister chan *Client
    broadcast  chan Event
    mu         sync.RWMutex
    online     map[string]int
}
```

Chapter 4 - Gin

What is it?

Gin is the HTTP framework that maps incoming requests to Go handler functions.

Why do we need it?

Without a router, the backend would need manual URL parsing and method checks. Gin gives clean route declarations, request context, JSON helpers, and middleware support.

How does it work?

You register routes like GET, POST, PATCH, and DELETE. Each route points to a handler. Middleware can run before handlers for logging, recovery, CORS, or admin checks.

How ChatSphere uses it

ChatSphere creates a router in NewRouter, registers /health, /ws, /api/v1/config, auth, users, profile, contacts, groups, messages, calls, statuses, upload, files, admin, and AI routes.

What would happen without it?

Every request would be harder to organize, and adding features would increase routing duplication.

What should I learn?

routes, handlers, middleware, path params, query params, request body parsing, JSON responses.

Where is it in the code?

• backend/internal/http/router.go

• backend/internal/http/ai.go

• backend/internal/http/email_auth.go

```
router.GET("/health", healthHandler)
api := router.Group("/api/v1")
registerMessageRoutes(api.Group("/messages"), dataStore, hub)
```

Gin idea	Meaning	Example
route	method plus path	GET /health
handler	request function	loginUser(dataStore)
middleware	shared logic	CORS, recovery
path param	URL variable	:id
query param	URL option	?limit=50
JSON response	structured output	gin.H{"status":"ok"}

Chapter 5 - REST API

What is it?

REST is a style for organizing HTTP APIs around resources and standard methods.

Why do we need it?

It makes backend behavior predictable. GET reads, POST creates or performs an action, PATCH updates part of a resource, DELETE removes or hides something.

How does it work?

The frontend calls endpoints. The backend decodes the request, performs work, and returns a status code and JSON response.

How ChatSphere uses it

ChatSphere uses GET for message lists, POST for sending, PATCH for editing, DELETE for Delete for Me and Delete for Everyone.

What would happen without it?

The API would become inconsistent. Developers would not know whether to POST, PATCH, or DELETE, and debugging would be harder.

What should I learn?

HTTP methods, path params, query params, body formats, status codes, and idempotency.

Where is it in the code?

• backend/internal/http/router.go

Method	Meaning	ChatSphere example	Purpose
GET	Read	GET /api/v1/messages/:recipientId	Load messages
POST	Create/action	POST /api/v1/messages	Send message
PATCH	Partial update	PATCH /api/v1/messages/:id	Edit body
DELETE	Remove/hide	DELETE /api/v1/messages/:id/me	Delete for me
DELETE	Tombstone	DELETE /api/v1/messages/:id/everyone	Delete for everyone

Why PATCH for edit?

Editing changes body and edited_at, not the whole message resource.

Chapter 6 - Databases

What is it?

A database is durable storage for application facts.

Why do we need it?

ChatSphere cannot keep users and messages only in variables because process memory disappears when the server restarts.

How does it work?

PostgreSQL stores data in tables. SQL statements insert, select, update, and delete rows. Indexes speed up common lookups.

How ChatSphere uses it

ChatSphere stores app_users, messages, groups, group_members, group_messages, reactions, deletions, stars, statuses, attachments, calls, reports, and AI usage.

What would happen without it?

Users would lose accounts and chat history after restart. Realtime delivery alone would not be enough because late/offline users need persisted history.

What should I learn?

tables, rows, columns, primary keys, indexes, unique constraints, NULL, timestamps, PostgreSQL basics.

Where is it in the code?

• backend/internal/store/store.go

Term	Definition	ChatSphere example
table	Collection of records	messages
row	One record	One message
column	One field	body
primary key	Unique ID	messages.id
index	Fast lookup	conversation_id, created_at, seq
unique	No duplicates	app_users.email
NULL	No value	edited_at before edit
timestamp	Time value	created_at

Chapter 7 - ChatSphere Database Design

What is it?

Database design is how tables are shaped and related.

Why do we need it?

Good design keeps different kinds of facts separate. A message is not the same thing as a reaction, a private deletion, or a private star.

How does it work?

Tables store core entities and relationship tables connect them. Separate tables model many-to-many or per-user state.

How ChatSphere uses it

ChatSphere uses messages for direct messages, group_messages for group messages, message_reactions for reactions, message_deletions for Delete for Me, and starred_messages for private stars.

What would happen without it?

A single giant messages table with many boolean columns would leak private state, duplicate data, and make group/direct behavior messy.

What should I learn?

ER diagrams, relationship tables, per-user state, constraints, and indexes.

Where is it in the code?

• backend/internal/store/store.go

```
app_users
  |        |
  |        +-- user_blocks, reports, ai_usage, call_history
  |
  +-- messages -- message_reactions
  |            \-- message_deletions
  |            \-- starred_messages
  |
  +-- statuses -- status_views

groups -- group_members -- app_users
  |
  +-- group_messages -- message_reactions
                     \-- message_deletions
                     \-- starred_messages
```

Table	Why it exists
app_users	Accounts, profile, password hash, avatar, blocked flag
messages	Direct message history
attachments	DB fallback bytes or Cloudinary metadata
groups	Group metadata
group_members	Membership and roles
group_messages	Group message history
message_reactions	One reaction per user per message
message_deletions	Delete for Me state
starred_messages	Private stars
statuses/status_views	Stories and viewer tracking
call_history	Persistent call records

Chapter 8 - SQL Through ChatSphere

What is it?

SQL is the language used to ask PostgreSQL for data changes and reads.

Why do we need it?

SQL lets the backend persist facts and retrieve the correct subset, such as one conversation, one group, or one user's starred messages.

How does it work?

INSERT creates rows, SELECT reads rows, UPDATE changes rows, DELETE removes rows, JOIN combines tables, WHERE filters, ORDER BY sorts, LIMIT caps result size.

How ChatSphere uses it

Store methods use SQL queries with parameters like $1 and $2. User input is passed separately from SQL text.

What would happen without it?

Without SQL fundamentals, store.go looks like magic and database bugs become guesswork.

What should I learn?

INSERT, SELECT, UPDATE, DELETE, JOIN, WHERE, ORDER BY, LIMIT, Scan, and SQL injection prevention.

Where is it in the code?

• backend/internal/store/store.go

SQL idea	Simplified ChatSphere operation
INSERT	Create a message row
SELECT	Load messages for a conversation
UPDATE	Set edited_at and new body
DELETE	Remove a group member row
JOIN	Load group members with app_users
WHERE	Require sender or group membership
ORDER BY	Sort by created_at and seq
LIMIT	Return only recent messages

```
rows, err := db.Query(ctx,
    "select id, body from messages where conversation_id=$1 limit $2",
    conversationID,
    limit,
)
```

SQL injection defense

Parameterized queries keep user text separate from SQL commands.

Chapter 9 - Store Layer

What is it?

The store layer is the persistence boundary.

Why do we need it?

Handlers should not become giant SQL files. A store method gives a named operation such as SaveMessage or DeleteMessageForEveryone.

How does it work?

Handlers call store methods. Store methods validate data relationships, run SQL, scan rows, and return Go structs.

How ChatSphere uses it

The HTTP and WebSocket paths both use store methods for durable operations.

What would happen without it?

SQL would be duplicated across routes and behavior could differ between HTTP and WebSocket paths.

What should I learn?

data access layers, Scan, QueryRow, Exec, transactions, repository patterns.

Where is it in the code?

• backend/internal/store/store.go

```
HTTP Handler
  |
  v
Store Method
  |
  v
SQL
  |
  v
PostgreSQL
```

Chapter 10 - Authentication

What is it?

Authentication asks: Who are you?

Why do we need it?

ChatSphere must know which user is sending messages, editing profiles, viewing groups, or calling another user.

How does it work?

Users verify email, complete profile/password, log in, receive a signed token, and send it on protected requests.

How ChatSphere uses it

requireUser reads Authorization: Bearer token and validates the HMAC-signed custom token.

What would happen without it?

Anyone could pretend to be anyone else.

What should I learn?

OTP, bcrypt, token signing, token expiry, auth middleware.

Where is it in the code?

• backend/internal/http/router.go

• backend/internal/http/email_auth.go

• backend/internal/store/store.go

```
Browser -> Login request -> Backend
Backend -> Find user -> bcrypt compare
Backend -> Generate HMAC token
Browser -> Authenticated requests with Bearer token
```

Chapter 11 - Password Security

What is it?

Password security means storing proof of a password without storing the password itself.

Why do we need it?

If a database leaks, plaintext passwords would immediately compromise users.

How does it work?

bcrypt creates a slow one-way hash. Login compares the candidate password to that hash.

How ChatSphere uses it

UpsertUser and UpdatePassword hash passwords. Authenticate compares hashes.

What would happen without it?

A leaked database would expose real passwords.

What should I learn?

hashing, salts, bcrypt cost, verification, why hashes are not decrypted.

Where is it in the code?

• backend/internal/store/store.go

```
password -> bcrypt hash -> stored
login attempt -> bcrypt compare -> match/fail
```

Chapter 12 - Tokens / HMAC

What is it?

A token is a portable proof that the backend authenticated a user.

Why do we need it?

The frontend needs to call protected APIs without sending the password every time.

How does it work?

ChatSphere creates base64url(userID|email|expiry|signature). The signature is HMAC-SHA256 over the payload using AUTH_SECRET.

How ChatSphere uses it

authUserFromToken verifies signature, expiry, and revocation.

What would happen without it?

Attackers could modify user IDs or forge tokens if no signature existed.

What should I learn?

HMAC, payload, secret, expiry, constant-time compare, secret rotation.

Where is it in the code?

• backend/internal/http/router.go

```
payload = userID|email|expires
signature = HMAC(payload, AUTH_SECRET)
token = base64url(payload|signature)
```

Chapter 13 - Authorization

What is it?

Authorization asks: What are you allowed to do?

Why do we need it?

Authentication alone is not enough. A real user still must not edit another user's message or read a private group.

How does it work?

Backend code checks sender ownership, group membership, block relationships, status ownership, admin tokens, and message accessibility.

How ChatSphere uses it

Store methods enforce many critical rules. Handlers translate forbidden actions to 403 or 404.

What would happen without it?

Frontend-only checks could be bypassed by changing browser requests.

What should I learn?

ownership, role checks, access control, trust boundaries.

Where is it in the code?

- backend/internal/http/router.go
- backend/internal/store/store.go

```
Frontend check = nicer UX
Backend check  = real security
```

Chapter 14 - Sending One Message

What is it?

Sending a message is the full backend path in miniature.

Why do we need it?

It combines UI, HTTP/WebSocket, auth, validation, persistence, ordering, response, and realtime delivery.

How does it work?

The frontend creates a draft/optimistic message, sends it, backend saves it, then broadcasts to participants.

How ChatSphere uses it

ChatSphere uses SaveMessage and chat.message. It also supports client IDs for idempotency.

What would happen without it?

Messages might duplicate, disappear, arrive late, or bypass permissions.

What should I learn?

request lifecycles, optimistic UI, idempotency, database insert, realtime broadcast.

Where is it in the code?

- frontend/src/components/AppShell.tsx
- backend/internal/http/router.go
- backend/internal/store/store.go
- backend/internal/realtime/client.go

Chapter 15 - Message Ordering

What is it?

Message ordering is the rule for deciding what appears first.

Why do we need it?

Rapid sends can produce timestamp ties. ORDER BY created_at alone can become unstable.

How does it work?

ChatSphere direct messages include a seq column and order by created_at plus seq for deterministic ordering.

How ChatSphere uses it

The frontend send queue and server ordering work together to keep rapid messages predictable.

What would happen without it?

Messages sent quickly could appear in different orders for different users.

What should I learn?

deterministic ordering, tie-breakers, rapid sends, race conditions.

Where is it in the code?

- backend/internal/store/store.go
- frontend/src/components/AppShell.tsx

```
created_at tie:
msg A 10:00:00.000
msg B 10:00:00.000

ORDER BY created_at, seq -> stable order
```

Chapter 16 - Optimistic UI

What is it?

Optimistic UI shows an action before the server fully confirms it.

Why do we need it?

Chat apps feel broken if every message waits visibly for upload, network, database, and ack.

How does it work?

Frontend inserts a sending message, then marks it sent or failed when the backend responds.

How ChatSphere uses it

ChatSphere has sending/uploading/sent/failed states and retry behavior.

What would happen without it?

The app would feel slow and mobile networks would be frustrating.

What should I learn?

temporary IDs, retries, failure states, reconciliation, idempotency.

Where is it in the code?

- frontend/src/components/AppShell.tsx
- frontend/src/components/chat/MessageBubble.tsx

```
User sends -> UI shows sending
Server confirms -> UI marks sent
Server fails -> UI marks failed + retry
```

Chapter 17 - WebSockets

What is it?

A WebSocket is a persistent two-way connection.

Why do we need it?

Chat recipients need new events immediately without polling every second.

How does it work?

A token-authenticated /ws connection creates client read and write pumps. The hub dispatches events.

How ChatSphere uses it

ChatSphere sends chat.message, group.message, typing, read, reaction, edit, delete, presence, and call signaling events.

What would happen without it?

The frontend would need repeated polling, wasting network and delaying updates.

What should I learn?

upgrade, persistent connection, readPump, writePump, ping/pong.

Where is it in the code?

• backend/internal/realtime/client.go

• backend/internal/realtime/hub.go

```
HTTP: request -> response -> done

WebSocket:
client <=================> server
```

Chapter 18 - ChatSphere Realtime Hub

What is it?

The hub is the central in-memory coordinator for WebSocket clients.

Why do we need it?

Many clients can connect and events need targeted delivery.

How does it work?

Clients register, unregister, and receive broadcasts through channels. The hub filters by TargetUserIDs.

How ChatSphere uses it

Hub tracks online counts, lastSeen, active calls, and connected clients.

What would happen without it?

Each connection would need to know about every other connection manually.

What should I learn?

hub pattern, targeted broadcasts, connection lifecycle.

Where is it in the code?

• backend/internal/realtime/hub.go

```
register -> clients map
unregister -> cleanup
broadcast -> dispatch -> client.send
```

Chapter 19 - Goroutines

What is it?

A goroutine is lightweight concurrent work in Go.

Why do we need it?

Realtime servers handle many users at the same time.

How does it work?

ChatSphere runs the hub, each client's readPump/writePump, and delayed cleanup in goroutines.

How ChatSphere uses it

go hub.Run(), go client.writePump(), and go client.readPump() are central to realtime behavior.

What would happen without it?

One slow socket could block other users.

What should I learn?

concurrency, scheduling, blocking, lifecycle cleanup.

Where is it in the code?

• backend/cmd/server/main.go

• backend/internal/realtime/client.go

```
go hub.Run()
go client.readPump()
go client.writePump()
```

Chapter 20 - Channels

What is it?

Channels are typed pipes between goroutines.

Why do we need it?

They let goroutines communicate without directly mutating each other's internals.

How does it work?

The hub uses register, unregister, broadcast, and client send channels.

How ChatSphere uses it

Broadcasting a realtime event means sending it into the hub broadcast channel.

What would happen without it?

Concurrent code would become tangled and less safe.

What should I learn?

channel send/receive, buffering, select.

Where is it in the code?

- backend/internal/realtime/hub.go
- backend/internal/realtime/client.go

```
Goroutine A -> channel -> Goroutine B
```

Chapter 21 - Mutexes And Race Conditions

What is it?

A mutex is a lock that protects shared data.

Why do we need it?

Go maps are unsafe for concurrent writes. Realtime maps are shared by many goroutines.

How does it work?

Hub.mu protects online, lastSeen, and calls. Other mutexes protect OTP codes, AI maps, token revocation, and JSON fallback data.

How ChatSphere uses it

ChatSphere uses sync.RWMutex for shared realtime maps.

What would happen without it?

The backend could crash or show wrong online/call state.

What should I learn?

race conditions, Lock, Unlock, RLock, RUnlock.

Where is it in the code?

• backend/internal/realtime/hub.go

• backend/internal/http/email_auth.go

• backend/internal/http/ai.go

```
mu.Lock()
modify shared map
mu.Unlock()
```

Chapter 22 - Presence

What is it?

Presence is online/offline/last-seen state.

Why do we need it?

Users expect to know if someone is available now.

How does it work?

Online count increments on WebSocket connect and decrements on disconnect. A 15 second grace period avoids flicker.

How ChatSphere uses it

User list and UI badges use hub presence data. Last seen is currently in memory.

What would happen without it?

Online badges and last-seen text would be unreliable or impossible.

What should I learn?

connection counts, last seen, reconnect grace, presence.updated.

Where is it in the code?

• backend/internal/realtime/hub.go

• backend/internal/http/router.go

• frontend/src/components/AppShell.tsx

```
connect -> online count +1
disconnect -> wait 15s -> lastSeen -> presence.updated
```

Chapter 23 - Typing Indicator

What is it?

Typing is temporary realtime state.

Why do we need it?

It only matters right now; storing it permanently would pollute the database.

How does it work?

Frontend sends start/stop intent and backend broadcasts typing.start or typing.stop to participants.

How ChatSphere uses it

ChatSphere handles typing via /api/v1/messages/typing and realtime events.

What would happen without it?

The app might store useless rows like user was typing yesterday.

What should I learn?

persistent vs ephemeral data, realtime UX.

Where is it in the code?

- backend/internal/http/router.go
- frontend/src/components/AppShell.tsx

```
start typing -> typing.start
stop/send -> typing.stop
no database row
```

Chapter 24 - Message Reactions

What is it?

Reactions are per-user emoji responses to messages.

Why do we need it?

Many users can react to one message, and each user should have at most one current reaction per message.

How does it work?

message_reactions has message_type, message_id, user_id, emoji, created_at with a unique key.

How ChatSphere uses it

ChatSphere toggles same emoji off and replaces a different emoji.

What would happen without it?

Duplicate reactions or leaked reaction state would be easy.

What should I learn?

many-to-one relationships, unique constraints, aggregation.

Where is it in the code?

• backend/internal/store/store.go

• backend/internal/http/router.go

```
message_reactions:
(direct, m1, userA, heart)
(direct, m1, userB, thumbs-up)
```

Chapter 25 - Reply / Quote

What is it?

A reply stores a reference to another message.

Why do we need it?

Copying original text would duplicate data and make deleted-original behavior harder.

How does it work?

reply_to_message_id points to the original. List methods resolve quote preview metadata.

How ChatSphere uses it

ChatSphere validates same conversation or same group before saving a reply.

What would happen without it?

Users could quote messages from private conversations or stale copied content could leak.

What should I learn?

references, validation, quote previews, unavailable originals.

Where is it in the code?

• backend/internal/store/store.go

• frontend/src/components/chat/MessageBubble.tsx

```
Message B
  reply_to_message_id -> Message A

List endpoint resolves a small quote preview
```

Chapter 26 - Edit Message

What is it?

Edit Message changes an existing message body.

Why do we need it?

Users need typo correction, but history/order should remain coherent.

How does it work?

PATCH validates the current user is sender, updates body, sets edited_at, and broadcasts message.edited.

How ChatSphere uses it

ChatSphere preserves created_at and attachments/replies/reactions while marking editedAt.

What would happen without it?

Edits could rewrite ownership or reorder messages incorrectly.

What should I learn?

PATCH, ownership, immutable createdAt, realtime patching.

Where is it in the code?

• backend/internal/http/router.go

• backend/internal/store/store.go

• frontend/src/components/chat/MessageComposer.tsx

```
PATCH /messages/:id
sender check
UPDATE body, edited_at
broadcast message.edited
```

Chapter 27 - Delete For Me Vs Delete For Everyone

What is it?

Delete for Me is private hiding. Delete for Everyone is a shared tombstone.

Why do we need it?

Users need both private cleanup and sender-controlled global removal.

How does it work?

Delete for Me writes message_deletions. Delete for Everyone sets deleted_for_everyone fields, hides body/attachment, clears reactions/stars, and broadcasts.

How ChatSphere uses it

ChatSphere keeps ordering and reply safety by not hard deleting normal messages for everyone.

What would happen without it?

Hard deletes would break replies, ordering, and realtime consistency.

What should I learn?

tombstones, private state, content scrubbing, authorization.

Where is it in the code?

• backend/internal/store/store.go

• backend/internal/http/router.go

• frontend/src/components/chat/MessageBubble.tsx

```
Delete for Me:
message row remains
message_deletions hides it for one user

Delete for Everyone:
message row becomes tombstone for all
```

Chapter 28 - Starred Messages

What is it?

A starred message is a private user bookmark.

Why do we need it?

One user starring a message should not make it starred for everyone.

How does it work?

starred_messages stores user_id, message_type, message_id, starred_at.

How ChatSphere uses it

ChatSphere lists direct and group starred messages separately and returns isStarred per viewer.

What would happen without it?

A messages.starred boolean would leak one user's preference to everyone.

What should I learn?

per-user state, privacy, listing endpoints.

Where is it in the code?

• backend/internal/store/store.go

• backend/internal/http/router.go

```
starred_messages:
user A -> message 1
user B -> message 9
```

Chapter 29 - Group Chat

What is it?

Group chat is messaging with membership and roles.

Why do we need it?

A group message goes to multiple users and must enforce group access.

How does it work?

groups stores metadata, group_members stores roles, group_messages stores content. Backend broadcasts to members.

How ChatSphere uses it

ChatSphere supports owner/admin/member roles and checks access server-side.

What would happen without it?

Outsiders could read or send messages by guessing group IDs.

What should I learn?

roles, membership, group authorization, fan-out.

Where is it in the code?

- backend/internal/store/store.go
- backend/internal/http/router.go

```
groups -> group_members -> app_users
   |
   v
group_messages -> broadcast to members
```

Chapter 30 - File Uploads

What is it?

Uploads send binary files using multipart/form-data.

Why do we need it?

JSON is not ideal for raw file bytes, and the backend must limit size and know file type.

How does it work?

Backend validates auth, file presence, max size, content type, then chooses Cloudinary or DB fallback.

How ChatSphere uses it

ChatSphere returns either a Cloudinary URL or attachment:id route.

What would happen without it?

Large or unsafe uploads could overload storage and network.

What should I learn?

multipart, MIME, file size, streaming, validation.

Where is it in the code?

• backend/internal/http/router.go

• backend/internal/store/store.go

• frontend/src/lib/mediaCompression.ts

```
file -> multipart upload -> validate -> Cloudinary or attachments table
```

Chapter 31 - Cloudinary

What is it?

Cloudinary is media storage plus CDN delivery.

Why do we need it?

Large media in PostgreSQL is usually not ideal for performance and cost.

How does it work?

Backend uploads to Cloudinary, stores secure URL/public ID, and may add delivery transformations.

How ChatSphere uses it

ChatSphere falls back to database storage only when Cloudinary is not configured.

What would happen without it?

Backend would serve every large file itself and database size could grow quickly.

What should I learn?

object storage, CDN, public ID, signed delete, transformations.

Where is it in the code?

• backend/internal/http/router.go

```
Backend -> Cloudinary upload API -> secure URL -> attachments metadata
```

Chapter 32 - Email OTP / Brevo

What is it?

OTP is a short-lived one-time code. Brevo is the email provider.

Why do we need it?

Email verification and password reset need a way to prove the user controls an email address.

How does it work?

Backend generates a crypto-random 6 digit code, stores it for 10 minutes, rate-limits requests, and sends through Brevo.

How ChatSphere uses it

ChatSphere stores OTP codes and request rates in memory.

What would happen without it?

Attackers could spam email and users could sign up with unverified addresses.

What should I learn?

OTP expiry, rate limiting, email APIs, in-memory limitations.

Where is it in the code?

• backend/internal/http/email_auth.go

```
request code -> rate limit -> random code -> Brevo email -> verify code
```

Chapter 33 - Rate Limiting

What is it?

Rate limiting caps how often a client can do something.

Why do we need it?

It protects the app from spam, brute force behavior, email abuse, and AI quota/cost abuse.

How does it work?

ChatSphere uses per-email OTP windows, AI cooldown, AI IP rate limit, and persistent daily AI usage.

How ChatSphere uses it

ReserveAIUsage protects daily Gemini quota even with concurrent requests.

What would happen without it?

One user or script could consume resources quickly.

What should I learn?

sliding windows, cooldowns, quotas, Redis alternatives.

Where is it in the code?

- backend/internal/http/email_auth.go
- backend/internal/http/ai.go
- backend/internal/store/store.go

```
request -> check rate state -> allow or 429 Too Many Requests
```

Chapter 34 - Gemini AI

What is it?

Gemini is the external AI model provider used by ChatSphere AI Assistant.

Why do we need it?

AI calls require a secret key, quota control, timeout, and safe errors.

How does it work?

Frontend sends prompt to backend. Backend validates auth/rates/quota, calls Gemini, and returns the response.

How ChatSphere uses it

ChatSphere keeps GEMINI_API_KEY on the backend and stores usage in ai_usage.

What would happen without it?

The key could leak and users could bypass quota if the frontend called Gemini directly.

What should I learn?

model, system instruction, prompt, quota, timeout, safe errors.

Where is it in the code?

- backend/internal/http/ai.go
- backend/internal/gemini/gemini.go
- frontend/src/components/AIChat.tsx

```
Frontend -> ChatSphere backend -> Gemini API -> backend -> frontend
```

Chapter 35 - CORS

What is it?

CORS is the browser's cross-origin request control.

Why do we need it?

Frontend and backend may run on different domains. Browser needs backend permission to read responses.

How does it work?

Gin CORS allows FRONTEND_ORIGIN, methods, Authorization, Content-Type, and credentials.

How ChatSphere uses it

ChatSphere configures CORS in NewRouter.

What would happen without it?

Browser fetch calls from the frontend domain could be blocked.

What should I learn?

origins, preflight, allowed methods, allowed headers.

Where is it in the code?

• backend/internal/http/router.go

• backend/internal/config/config.go

```
frontend domain -> backend domain
browser checks Access-Control-Allow-Origin
```

Chapter 36 - WebRTC

What is it?

WebRTC is browser technology for realtime audio/video media.

Why do we need it?

WebSocket is good for signaling and chat events; it is not the ideal path for live camera/microphone streams.

How does it work?

Browsers create RTCPeerConnection, exchange offer/answer/ICE via signaling, then send media tracks.

How ChatSphere uses it

ChatSphere uses WebSocket for call_offer, call_answer, and candidates, while media flows through WebRTC.

What would happen without it?

All media would need to go through the backend or calls would not connect.

What should I learn?

RTCPeerConnection, offer, answer, ICE candidates, media tracks.

Where is it in the code?

• frontend/src/components/AudioCall.tsx

• backend/internal/realtime/client.go

```
WebSocket = signaling
WebRTC    = audio/video media
```

Chapter 37 - STUN / TURN / ICE

What is it?

STUN, TURN, and ICE help WebRTC connect across NATs and firewalls.

Why do we need it?

Two devices often cannot directly reach each other on private networks.

How does it work?

STUN discovers public connection info. TURN relays media when direct paths fail. ICE tests routes and chooses one.

How ChatSphere uses it

ChatSphere uses Google STUN and optional Metered TURN credentials.

What would happen without it?

Calls would fail on many real-world networks.

What should I learn?

NAT, firewall traversal, relay, ICE candidate selection.

Where is it in the code?

• frontend/src/lib/webrtcConfig.ts

```
Browser A behind NAT <-> STUN/TURN/ICE <-> Browser B behind NAT
```

Chapter 38 - Call Flow

What is it?

A call flow is the sequence of signaling and media setup steps.

Why do we need it?

Calls need coordination before media can flow.

How does it work?

Caller sends call_offer, callee sends call_answer, both exchange ICE candidates, then WebRTC media connects.

How ChatSphere uses it

Backend validates call state, busy state, block relationships, and participant membership in active calls.

What would happen without it?

Users would ring without authorization or calls would not establish.

What should I learn?

signaling events, call sessions, call history, TURN fallback.

Where is it in the code?

- backend/internal/realtime/client.go
- backend/internal/realtime/hub.go
- backend/internal/store/store.go
- frontend/src/components/AudioCall.tsx

```
User A -> call_offer -> backend -> User B
User B -> call_answer -> backend -> User A
ICE exchange -> WebRTC media
```

Chapter 39 - Environment Variables

What is it?

Environment variables configure behavior outside the code.

Why do we need it?

Secrets and environment-specific values must not be hard-coded.

How does it work?

Config.Load reads variables such as DATABASE_URL, AUTH_SECRET, CLOUDINARY_*, BREVO_*, GEMINI_*, FRONTEND_ORIGIN, upload limits, and AI limits.

How ChatSphere uses it

ChatSphere .env.example documents names without real secrets.

What would happen without it?

Secrets could leak in source control and deployments would be hard to change.

What should I learn?

code vs config vs secrets, secret rotation, deployment envs.

Where is it in the code?

- backend/internal/config/config.go
- backend/.env.example
- frontend/.env.example

```
code + environment variables -> configured running app
```

Chapter 40 - Deployment

What is it?

Deployment is running the built application on hosted infrastructure.

Why do we need it?

Users need public HTTPS frontend, backend, database, WebSocket, and external service access.

How does it work?

Frontend hosting serves Next.js. Backend hosting runs Go. PostgreSQL stores data. HTTPS/WSS secure traffic.

How ChatSphere uses it

ChatSphere has Railway backend config, Netlify frontend config, and an Android WebView pointing at the hosted web app.

What would happen without it?

Local-only apps would not be available to real users.

What should I learn?

builds, env vars, HTTPS, WSS, health checks, cold starts.

Where is it in the code?

- backend/railway.json
- netlify.toml
- frontend/netlify.toml
- ChatSphere_Android/app/src/main/java/com/chatsphere/app/MainActivity.kt

```
Browser/Android -> frontend hosting -> HTTPS/WSS backend -> PostgreSQL + services
```

Chapter 41 - Security

What is it?

Security is protecting users, data, secrets, and infrastructure.

Why do we need it?

Chat apps carry private messages, accounts, files, calls, and third-party API keys.

How does it work?

ChatSphere uses bcrypt, HMAC tokens, authorization checks, parameterized SQL, rate limits, upload limits, CORS, and backend-only secrets.

How ChatSphere uses it

Current production review areas include WebSocket origin, in-memory revocation/OTP, admin model, uploads, and foreign keys.

What would happen without it?

Users could impersonate others, leak secrets, inject SQL, or abuse services.

What should I learn?

attack thinking, defense layers, least privilege, validation, monitoring.

Where is it in the code?

- backend/internal/http/router.go
- backend/internal/store/store.go
- backend/internal/realtime/client.go

```
ATTACK -> backend defense -> safe failure
```

Chapter 42 - Testing

What is it?

Testing is automated checking that code behaves as expected.

Why do we need it?

Backends need tests because bugs can leak data or break core workflows.

How does it work?

go test ./... runs tests in all backend packages. httptest can call handlers without a real browser.

How ChatSphere uses it

ChatSphere tests health, AI limits, Gemini request shape, messaging, reactions, replies, edits, deletes, stars, groups, statuses, calls, uploads, and idempotency.

What would happen without it?

Manual clicking would miss regressions and authorization bugs.

What should I learn?

unit tests, integration tests, httptest, fixtures, fake services.

Where is it in the code?

- backend/internal/http/*_test.go
- backend/internal/gemini/gemini_test.go

```
go test ./... -> run package tests -> pass/fail output
```

Chapter 43 - Debugging

What is it?

Debugging is tracing evidence from symptom to cause.

Why do we need it?

Backend bugs cross frontend, network, auth, handler, store, SQL, logs, and realtime events.

How does it work?

Start with the request or event, inspect status code/body, then backend logs and store behavior.

How ChatSphere uses it

For a failed send, inspect composer state, network response, auth token, handler validation, SaveMessage, SQL, WebSocket ack.

What would happen without it?

Guessing wastes time and can create unrelated fixes.

What should I learn?

network tab, logs, status codes, SQL errors, event tracing.

Where is it in the code?

- frontend/src/components/AppShell.tsx
- backend/internal/http/router.go
- backend/internal/store/store.go
- backend/internal/realtime/client.go

```
Symptom -> Network -> Handler -> Store -> SQL -> WebSocket -> UI
```

Chapter 44 - What ChatSphere Taught Me

What is it?

ChatSphere is a real backend learning project, not just CRUD.

Why do we need it?

It combines durable data, realtime events, authentication, authorization, media, calls, AI, email, and deployment.

How does it work?

Professional backends are systems of cooperating parts. Each part has a responsibility and a failure mode.

How ChatSphere uses it

ChatSphere demonstrates data modeling, store layer, WebSocket hub, WebRTC signaling, external APIs, and security boundaries.

What would happen without it?

You would miss the bigger engineering lessons if you only studied isolated features.

What should I learn?

system thinking, tradeoffs, production review, refactoring paths.

Where is it in the code?

- All backend files and key frontend integration files

```
CRUD + realtime + concurrency + media + auth + WebRTC + external APIs
```

Chapter 45 - Complete Backend Learning Roadmap

What is it?

A roadmap is an ordered learning plan.

Why do we need it?

Backend development is easier when foundations come before advanced realtime and production topics.

How does it work?

Study one topic, inspect the matching ChatSphere files, then complete a small practical exercise.

How ChatSphere uses it

This roadmap maps every topic back to the project.

What would happen without it?

Random study can leave gaps that make advanced features confusing.

What should I learn?

Follow the order, but revisit earlier chapters while debugging real code.

Where is it in the code?

• backend/

• frontend/src/components/

• frontend/src/lib/

Topic	Difficulty	Time	Understand	Files	Exercise
HTTP	Easy	1-2 days	Requests, responses, headers, status codes	router.go	Trace /health and /messages.
REST	Easy	2 days	GET/POST/PATCH/DELETE resources	router.go	Make endpoint table.
JSON	Easy	1 day	Objects, arrays, request bodies	router.go	Write sample payloads.
Go	Medium	1-2 weeks	Structs, errors, methods	store.go	Explain User and Message.
Gin	Medium	3-5 days	Routes and handlers	router.go	Trace login.
SQL/PostgreSQL	Medium	1-2 weeks	Tables, indexes, queries	store.go	Draw schema.
Authentication	Medium	1 week	OTP, bcrypt, token	router.go, email_auth.go	Explain login flow.
Authorization	Medium	1 week	Ownership and roles	store.go	Test non-member access.
WebSockets	Hard	1-2 weeks	Persistent events	client.go	Trace message_send.
Go concurrency	Hard	2 weeks	goroutines, channels, mutexes	hub.go	Draw hub lifecycle.
Uploads/cloud	Medium	1 week	multipart and CDN	router.go	Upload image flow.
WebRTC	Hard	2-3 weeks	offer, answer, ICE	AudioCall.tsx	Draw call flow.
Security	Hard	ongoing	attacks and defenses	many files	Make hardening checklist.
Testing	Medium	1 week	go test, httptest	*_test.go	Read one test.
Deployment	Medium	1 week	env, HTTPS, WSS	railway/netlify	List prod envs.
Scaling	Hard	2-4 weeks	shared state, Redis	hub.go	Externalize in-memory maps.

Backend Cheat Sheet

What is it?

A cheat sheet is a compressed reference for revision.

Why do we need it?

It helps you recall the major backend concepts quickly before interviews or coding sessions.

How does it work?

Read the left column, cover the right column, and explain the meaning from memory.

How ChatSphere uses it

The terms here are all used by ChatSphere.

What would happen without it?

You would need to reread whole chapters every time.

What should I learn?

Use it for fast revision after you understand the chapters.

Where is it in the code?

• This PDF

• CHATSPHERE_BACKEND_ARCHITECTURE_AND_LEARNING_GUIDE.md

Concept	Remember
Backend	Server-side authority for data and security
Handler	Receives request and calls store
Store	Persistence and data rules
WebSocket	Persistent realtime connection
Tombstone	Deleted placeholder preserving order
HMAC token	Signed payload plus expiry
bcrypt	Slow one-way password hash
Parameterized SQL	Values separate from SQL text
Presence	Derived from WebSocket connection count
TURN	Media relay for difficult networks

Glossary

What is it?

A glossary defines important terms.

Why do we need it?

Backend words can sound harder than they are. Clear definitions make the code easier to read.

How does it work?

Use this section when a chapter introduces a term you want to refresh.

How ChatSphere uses it

Every term here appears in or around ChatSphere.

What would happen without it?

Undefined vocabulary makes technical reading slow.

What should I learn?

Review these terms until you can explain them with a ChatSphere example.

Where is it in the code?

• This PDF

• backend source files

Term	Meaning
API	Contract other code can call
Endpoint	Method plus path
HTTP	Request/response protocol
REST	Resource-oriented API style
JSON	Structured text data
Middleware	Shared logic around handlers
Route	Mapping to a handler
Database	Durable storage
SQL	Relational query language
Index	Lookup accelerator
Primary key	Unique row ID
Unique constraint	No duplicate combination
WebSocket	Persistent two-way connection
Goroutine	Concurrent Go work
Channel	Go communication pipe
Mutex	Lock for shared state
Race condition	Unsafe timing bug
WebRTC	Browser realtime media
ICE	Route discovery process
STUN	Public network discovery
TURN	Media relay
CDN	Distributed content delivery
CORS	Browser origin control

Term	Meaning
Rate limit	Cap on repeated actions
Idempotency	Safe repeated operation
Optimistic UI	Show before confirmation
Authentication	Who are you?
Authorization	What can you do?
Tombstone	Deleted placeholder

Interview Questions

What is it?

Interview questions test whether you can explain the concepts, not just name them.

Why do we need it?

ChatSphere gives you concrete examples for each backend idea.

How does it work?

Read each question, answer from memory, then compare with the short answer.

How ChatSphere uses it

These answers are based on current ChatSphere architecture.

What would happen without it?

You might memorize terms without understanding how they apply.

What should I learn?

Practice explaining each answer with a source file reference.

Where is it in the code?

• backend/internal/http/router.go

• backend/internal/store/store.go

• backend/internal/realtime/hub.go

Question	Short answer
What is a backend?	The server-side authority for data, security, and services.
Why use REST?	It gives predictable HTTP resource operations.
Why use PATCH for edit?	Only part of the message changes.
Why use WebSocket?	To push realtime events without polling.
What is a mutex?	A lock protecting shared state.
What does HMAC do?	Proves a token payload was signed by the backend secret.
What is idempotency?	Repeating a request does not create duplicate effects.
Why is TURN needed?	Some networks block direct WebRTC media paths.
Why store Delete for Me separately?	It is private per user.
Why use an index?	To speed common database lookups and ordering.
What is optimistic UI?	Showing a change before server confirmation.
Why keep API secrets on backend?	Frontend code is visible to users.
Why use bcrypt?	To store a slow one-way password hash.
Why not trust frontend authorization?	Users can modify browser requests.
What should scale reviews focus on?	In-memory presence, calls, OTPs, revocation, and rate maps.

Recommended Study Order

What is it?

The final study order is the shortest path from zero backend clarity to understanding ChatSphere's advanced features.

Why do we need it?

Each topic builds on the previous topic.

How does it work?

Start with HTTP, REST, JSON, and Go. Then move to Gin, SQL, PostgreSQL, auth, authorization, WebSockets, concurrency, uploads, cloud services, WebRTC, security, testing, deployment, and scaling.

How ChatSphere uses it

The roadmap chapter maps each topic to files and exercises.

What would happen without it?

Jumping straight into WebRTC or WebSockets can feel overwhelming without HTTP, Go, and auth fundamentals.

What should I learn?

After each topic, open the referenced source file and explain one real function out loud.

Where is it in the code?

- This PDF roadmap
- backend source files

Best practice

Documentation becomes skill when you trace real code after reading the explanation.